ECSCI24302 – Machine Learning Essentials

# Project File Guide

The file tree of the AI vs Human Text Classifier: what each file does, and the code written in it

| 47 | 34 | 67 | 3 |
| --- | --- | --- | --- |
| project files explained | files with content printed in full | notebook cells mapped | parts: training, backend, frontend |

# Contents

1. How the Project Fits Together 3

2. The File Tree 4

3. Root Files 5

4. model_training: How the Model Was Made 7

5. saved models: The Trained Model 13

6. backend: The API Server 14

7. frontend: The Web Interface 23

8. docs: Project Documents 37

9. What Happens When You Click Analyze 38

Code is printed exactly as it appears in the project files, with line numbers. Long lines wrap. Not listed here: `.venv` and `node_modules` (installed packages, recreated by the setup commands), `.git` (Git's own records) and `__pycache__` (temporary Python files).

# 1. How the Project Fits Together

The project has three parts. **Training** happens once in a notebook and produces the model files. The **backend** loads those files and answers prediction requests. The **frontend** is the page people use.

**1. Training**
The Kaggle notebook creates the final model

→

**2. Saved model**
joblib model, clean_text.py and config JSON

→

**3. Backend**
FastAPI loads the model and serves the API

→

**4. Frontend**
React sends text and shows the result

| Folder | Language | Role |
|---|---|---|
| `model_training/` | Python, Jupyter | How the model was made. Not used while the app runs |
| `saved models/` | Binary, Python, JSON | The trained model and the two files it needs |
| `backend/` | Python, FastAPI | Loads the model and provides `/predict` and `/health` |
| `frontend/` | JavaScript, React, Tailwind | The dashboard in the browser |
| `docs/` | PDF, PowerPoint, Markdown, HTML | Brief, report and presentation |
| Root files | Batch, Markdown, CSV | Start script, instructions and earlier metrics |

**The key idea:** the frontend never touches the model. It only sends text to the backend over HTTP, and the backend is the only part that runs Machine Learning code.

# 2. The File Tree

| | |
|---|---|
| **New folder/** | Project root |
| ├── **backend/** | Python API server |
| │ ├── **app/** | The server's code |
| │ │ ├── __init__.py | Marks app as a Python package |
| │ │ ├── config.py | Reads settings from .env |
| │ │ ├── main.py | The FastAPI web server |
| │ │ └── model_service.py | Loads the model and predicts |
| │ ├── **models/** | Optional model location |
| │ │ └── README.txt | Note about this optional folder |
| │ ├── **tests/** | Automated tests |
| │ │ └── test_api.py | Automated backend tests |
| │ ├── .env | Your actual backend settings |
| │ ├── .env.example | Template for backend settings |
| │ └── requirements.txt | Python packages to install |
| ├── **docs/** | Brief, report and slides |
| │ ├── **archive/** | Older versions |
| │ │ ├── AI_vs_Human_Text_Classifier_full_23_slides.pptx | Older, longer presentation |
| │ │ └── pitch_full_with_viva_QA.md | Long pitch with viva answers |
| │ ├── AI_vs_Human_Text_Classifier.pptx | 8-slide presentation |
| │ ├── AI_vs_Human_Text_Classifier_Report.html | Same report as a web page |
| │ ├── AI_vs_Human_Text_Classifier_Report.pdf | Full printable report |
| │ ├── MLE_PBL_Project_Instructions.pdf | The course project brief |
| │ └── pitch.md | Text content of the 8 slides |
| ├── **frontend/** | React web interface |
| │ ├── **public/** | Static files |
| │ │ └── favicon.svg | Browser tab icon |
| │ ├── **src/** | The interface's code |
| │ │ ├── **components/** | Pieces of the page |
| │ │ │ ├── AboutModel.jsx | Explains the model simply |
| │ │ │ ├── Header.jsx | Title and status badge |
| │ │ │ ├── HistoryPanel.jsx | List of earlier analyses |
| │ │ │ ├── ProbabilityBar.jsx | One labelled percentage bar |
| │ │ │ ├── ResultCard.jsx | Prediction, bars and disclaimer |
| │ │ │ ├── SetupBanner.jsx | Setup error message |
| │ │ │ └── TextInput.jsx | Text box and buttons |
| │ │ ├── **lib/** | Helper code |
| │ │ │ ├── api.js | Talks to the backend |
| │ │ │ ├── samples.js | The two sample texts |
| │ │ │ ├── text.js | Small text helpers |
| │ │ │ └── useSessionHistory.js | Keeps history for this tab |
| │ │ ├── App.jsx | Main page and app state |
| │ │ ├── index.css | Global styles |
| │ │ └── main.jsx | Starts the React app |
| │ ├── .env.example | Template for the backend URL |
| │ ├── index.html | The page React loads into |
| │ ├── package-lock.json | Exact dependency versions (generated) |
| │ ├── package.json | JavaScript packages and scripts |
| │ ├── postcss.config.js | CSS processing plugins |
| │ ├── tailwind.config.js | Theme colours and fonts |
| │ └── vite.config.js | Development server settings |
| ├── **model_training/** | How the model was made |
| │ └── notebookc4a5d79b4f.ipynb | Complete model training notebook |
| ├── **saved models/** | The trained model |
| │ ├── ai_text_detector_model.zip | Zip of the three files above |
| │ ├── clean_text.py | Text cleaning used by the model |
| │ ├── improved_lr_pipeline.joblib | The trained final model (binary) |
| │ └── improved_model_config.json | Threshold and minimum length |
| ├── .gitignore | Files Git should not track |
| ├── final_model_comparison.csv | Metrics for the earlier models |
| ├── README.md | Setup and run instructions |
| └── run.bat | One-click start on Windows |

# 3. Root Files

### `README.md`

7.9 KB

The project's manual. It explains the model files, how to install and run the backend and frontend, the API, the configuration options and how the model works. It is documentation only and contains no running code.

### `run.bat`

2.1 KB · 58 lines

A Windows batch script. Double-clicking it sets up anything missing and starts the whole application.

- Checks that `python` and `npm` are on the PATH, and stops with a clear message if not.
- On first run, creates `backend\.venv`, installs `requirements.txt` and runs `npm install`.
- Copies `.env.example` to `.env` if needed, and warns that `MODEL_PATH` must be set.
- Opens two windows for the servers on ports 8000 and 5173, waits about 5 seconds using `ping`, then opens the browser.

```
@echo off
rem Starts the FastAPI backend and the Vite frontend in separate windows.
rem First run also creates the Python venv, installs dependencies and creates backend\.env.
setlocal
cd /d "%~dp0"

set "BACKEND=%~dp0backend"
set "FRONTEND=%~dp0frontend"
set "VENV_PY=%BACKEND%\.venv\Scripts\python.exe"

where python >nul 2>&1 || (echo [ERROR] Python not found on PATH. Install Python 3.10+ first. & goto :fail)
where npm >nul 2>&1 || (echo [ERROR] npm not found on PATH. Install Node.js 18+ first. & goto :fail)

rem --- Backend setup ---------------------------------------------------------
if not exist "%VENV_PY%" (
    echo Creating Python virtual environment in backend\.venv ...
    python -m venv "%BACKEND%\.venv" || goto :fail
    echo Installing backend dependencies ...
    "%VENV_PY%" -m pip install -r "%BACKEND%\requirements.txt" || goto :fail
)

if not exist "%BACKEND%\.env" (
    copy "%BACKEND%\.env.example" "%BACKEND%\.env" >nul
    echo.
    echo [WARNING] Created backend\.env from .env.example.
    echo           Set MODEL_PATH in backend\.env to your tuned_logistic_regression_pipeline.joblib
    echo           file. Until then the app will show a "Setup required" banner.
    echo.
)

rem --- Frontend setup --------------------------------------------------------
if not exist "%FRONTEND%\node_modules" (
    echo Installing frontend dependencies ...
    pushd "%FRONTEND%"
    call npm install || (popd & goto :fail)
    popd
)

rem --- Start both servers ----------------------------------------------------
echo Starting backend on http://localhost:8000 ...
start "AI Classifier - Backend" /D "%BACKEND%" cmd /k ".venv\Scripts\python.exe -m uvicorn app.main:app --
reload --port 8000"

echo Starting frontend on http://localhost:5173 ...
start "AI Classifier - Frontend" /D "%FRONTEND%" cmd /k "npm run dev"

rem Give the servers a moment, then open the app.
```

```
ping -n 6 127.0.0.1 >nul
start "" http://localhost:5173

echo.
echo Both servers are running in their own windows. Close those windows to stop them.
exit /b 0

:fail
echo.
echo Setup failed. See the messages above.
pause
exit /b 1
```

## `.gitignore`

172 bytes · 14 lines

Tells Git what to leave out of the repository: the virtual environment, `node_modules`, build output, private `.env` files and large data. Because `*.csv` and `*.joblib` are ignored, a fresh clone from GitHub does not include the model or the comparison table. They must be copied in separately.

```
# Python
backend/.venv/
__pycache__/
.pytest_cache/
backend/.env

# Node
frontend/node_modules/
frontend/dist/
frontend/.env

# Large data / model artifacts
*.csv
*.joblib
```

## `final_model_comparison.csv`

255 bytes · 5 lines

A small table of test results saved by the notebook (cell 53): accuracy, macro precision, recall, F1 and ROC-AUC for the baseline, Naive Bayes, tuned Logistic Regression and tuned Linear SVM. The report uses it. The app does not read it, and it does not include the final model.

```
Model,Accuracy,Macro Precision,Macro Recall,Macro F1,ROC-AUC
Dummy Baseline,0.6276,,,,
Multinomial Naive Bayes,0.9792,0.9793,0.9762,0.9777,0.9976
Tuned Logistic Regression,0.997,0.9974,0.9962,0.9968,0.9999
Tuned Linear SVM,0.9995,0.9995,0.9993,0.9994,1.0
```

# 4. model_training: How the Model Was Made

### `model_training/notebookc4a5d79b4f.ipynb`

305.5 KB

A Jupyter notebook, run on Kaggle, holding the whole Machine Learning workflow in 67 cells. It downloads the data, analyses and cleans it, trains and compares models, finds their weaknesses, trains the final model and exports it. The app never runs this file. It only uses the files the notebook produced. The map below shows what each group of cells does, followed by the most important code.

## Map of the notebook

| Cells | What they do |
|---|---|
| 0 | Project objective, datasets and workflow (notes) |
| 1–3 | Download the dataset, look at its first rows, check quality and labels |
| 4–11 | Target definition and EDA: 20,000-record sample, class balance, word and character counts, charts |
| 12–14 | Duplicate check over all rows using SHA-256 hashes; label validation |
| 15–19 | Read raw examples, find 4 blank rows, create and verify the cleaned CSV |
| 20–21 | Stratified 70:30 train-test split and checks |
| 22 | Baseline: DummyClassifier, always predicts human |
| 23–24 | TF-IDF features, with an explanation |
| 25–27 | Train Naive Bayes, Logistic Regression and Linear SVM |
| 28–29 | Tune Logistic Regression and Linear SVM with 3-fold cross-validation |
| 30, 53 | Compare models and save `final_model_comparison.csv` |
| 31–40 | Error analysis: misclassified texts, text patterns, and texts with conflicting labels |
| 41–44 | Read the strongest SVM features and interpret them |
| 45–47 | Prediction demo, with notes reporting the final results |
| 48–52, 54–55 | Save, reload and verify the earlier models; simple prediction function |
| 56–58 | Improvement: subsamples and snippets, train old and new recipes, first threshold and comparison |
| 59–61 | Three-outcome prediction function, threshold sweep and dry test on known AI and human text |
| 62–64 | Add blogs and BBC news, retrain as final model v2, compare and save |
| 65–66 | Print the final dry test; export the model files and zip for download |

## Most important code

These seven cells, printed exactly as written, carry the core of the training. The map above shows where to find the rest of the notebook.

## Cell 20

Split the data 70:30, keeping the class ratio in both parts.

```
# Step 9.1: Create 70:30 training and testing sets

import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv(cleaned_csv_path)

```

```
print("Dataset loaded:", df.shape)

X = df["text"]
y = df["generated"].astype("int8")

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("\nTRAINING AND TESTING SPLIT")
print("-" * 40)
print("Training records:", f"{len(X_train):,}")
print("Testing records:", f"{len(X_test):,}")

print("\nTraining class proportions:")
print(
    y_train.value_counts(normalize=True)
    .sort_index()
    .rename({0: "Human-written", 1: "AI-generated"})
    .mul(100).round(2).astype(str) + "%"
)

print("\nTesting class proportions:")
print(
    y_test.value_counts(normalize=True)
    .sort_index()
    .rename({0: "Human-written", 1: "AI-generated"})
    .mul(100).round(2).astype(str) + "%"
)
```

## Cell 23

Convert text to TF-IDF numbers. The vocabulary is learned from training data only.

```
# Step 11: TF-IDF Feature Extraction

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

# Create the TF-IDF vectorizer
tfidf_vectorizer = TfidfVectorizer(
    max_features=50000,
    stop_words="english",
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.95,
    sublinear_tf=True,
    dtype=np.float32
)

# Learn vocabulary from training data and transform it
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)

# Transform test data using the same vocabulary
X_test_tfidf = tfidf_vectorizer.transform(X_test)

print("TF-IDF FEATURE EXTRACTION")
print("-" * 40)
print("Training matrix shape:", X_train_tfidf.shape)
print("Testing matrix shape:", X_test_tfidf.shape)
print("Number of TF-IDF features:", len(tfidf_vectorizer.get_feature_names_out()))

print("\nTraining matrix storage format:", X_train_tfidf.format)
print("Training matrix density:", f"{X_train_tfidf.nnz / (X_train_tfidf.shape[0] * X_train_tfidf.shape[1]) *
100:.4f}%")
```

```
print("Test matrix density:", f"{X_test_tfidf.nnz / (X_test_tfidf.shape[0] * X_test_tfidf.shape[1]) * 100:.4f}%")
```

## Cell 28

Tune Logistic Regression. TF-IDF is inside the Pipeline so each cross-validation fold learns its own vocabulary, which prevents leakage.

```
# Step 15: Tune Logistic Regression using 3-fold cross-validation

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, StratifiedKFold

# Build a leakage-safe pipeline
lr_pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(
        max_features=30000,
        stop_words="english",
        ngram_range=(1, 2),
        min_df=2,
        max_df=0.95,
        sublinear_tf=True
    )),
    ("classifier", LogisticRegression(
        max_iter=1000,
        solver="liblinear",
        random_state=42
    ))
])

# Cross-validation setup
cv = StratifiedKFold(
    n_splits=3,
    shuffle=True,
    random_state=42
)

# Tune the regularization parameter
lr_param_grid = {
    "classifier__C": [0.5, 1.0]
}

lr_grid_search = GridSearchCV(
    estimator=lr_pipeline,
    param_grid=lr_param_grid,
    scoring="f1_macro",
    cv=cv,
    n_jobs=1,
    verbose=2,
    refit=True
)

print("Starting Logistic Regression tuning...")
print("This may take a while because the dataset is large.")

lr_grid_search.fit(X_train, y_train)

print("\nLOGISTIC REGRESSION TUNING RESULTS")
print("-" * 40)
print("Best parameters:", lr_grid_search.best_params_)
print("Best mean CV macro F1:", round(lr_grid_search.best_score_, 4))

print("\nAll cross-validation results:")
for mean_score, params in zip(
    lr_grid_search.cv_results_["mean_test_score"],
    lr_grid_search.cv_results_["params"]
):
    print(f"Macro F1: {mean_score:.4f} | Parameters: {params}")
```

## Cell 56

Explains why the earlier model flagged human text as AI. Creates the subsamples and the random 15–150-word snippets.

```python
# Step 20: Fixing false "AI" flags on human-written text
# ------------------------------------------------------------------------
# Why the earlier model flags human text as AI:
#  1. stop_words="english" removed function words (I, the, because, but ...),
#     which carry most of the writing-style signal. The model leaned on topic /
#     formal words such as "additionally", "essential", "important" instead.
#  2. It was trained only on full essays (~400 words) but is used on short
#     sentences it has never seen, so it gives overconfident guesses.
#  3. A fixed 0.5 threshold turns anything slightly formal into "AI".
# Fixes below: keep function words + add character n-grams (style features),
# train on short snippets as well as full essays, and choose the AI threshold
# on validation data so that very few human texts get flagged.

import os, re, gc
import numpy as np
import pandas as pd
import kagglehub
from sklearn.model_selection import train_test_split

if "df" not in globals():
    data_dir = kagglehub.dataset_download("shanegerami/ai-vs-human-text")
    df = pd.read_csv(os.path.join(data_dir, "AI_Human.csv"))
    df = df[df["text"].astype(str).str.strip().ne("")].reset_index(drop=True)
df["generated"] = df["generated"].astype(int)

# Same 70:30 split as Step 9.1, so the test part is never used for training
X_tr_all, X_te_all, y_tr_all, y_te_all = train_test_split(
    df["text"], df["generated"], test_size=0.30, random_state=42, stratify=df["generated"]
)

# Subsample to keep memory and time manageable on a free Colab runtime
X_tr, X_val, y_tr, y_val = train_test_split(
    X_tr_all, y_tr_all, train_size=40_000, test_size=15_000, random_state=42, stratify=y_tr_all
)
X_te, _, y_te, _ = train_test_split(
    X_te_all, y_te_all, train_size=20_000, random_state=42, stratify=y_te_all
)
X_tr, X_val, X_te = X_tr.astype(str), X_val.astype(str), X_te.astype(str)
del df, X_tr_all, X_te_all, y_tr_all, y_te_all
gc.collect()

rng = np.random.default_rng(42)

def random_snippet(text, min_words=15, max_words=150):
    """Cut a random passage of 15-150 words, to mimic short user inputs."""
    words = str(text).split()
    if len(words) <= min_words:
        return " ".join(words)
    length = int(rng.integers(min_words, min(max_words, len(words)) + 1))
    start = int(rng.integers(0, len(words) - length + 1))
    return " ".join(words[start:start + length])

def add_snippets(X, y):
    snippets = X.map(random_snippet)
    return (pd.concat([X, snippets], ignore_index=True),
            pd.concat([y, y], ignore_index=True))

X_tr_aug, y_tr_aug = add_snippets(X_tr, y_tr)
X_val_aug, y_val_aug = add_snippets(X_val, y_val)
X_te_short = X_te.map(random_snippet)          # short-text version of the test set

print("Training rows (essays + snippets):", f"{len(X_tr_aug):,}")
print("Validation rows (essays + snippets):", f"{len(X_val_aug):,}")
print("Test rows (full essays):", f"{len(X_te):,}")
print("Test rows (short snippets):", f"{len(X_te_short):,}")
```

## Cell 57

Defines clean_text() and the improved word and character recipe.

```
# Step 21: Train the old recipe and the improved recipe on the same data
import time
from sklearn.pipeline import Pipeline, FeatureUnion
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

def clean_text(text):
    text = str(text).lower()
    for a, b in {"’": "'", "‘": "'", "“": '"', "”": '"'}.items():
        text = text.replace(a, b)
    return re.sub(r"\s+", " ", text).strip()

# Old recipe (same settings as Step 15), trained on full essays only
old_style_model = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=30000, stop_words="english", ngram_range=(1, 2),
                              min_df=2, max_df=0.95, sublinear_tf=True)),
    ("classifier", LogisticRegression(C=1.0, max_iter=1000, solver="liblinear", random_state=42)),
])

# Improved recipe: function words + punctuation + character n-grams, more regularisation
improved_model = Pipeline([
    ("features", FeatureUnion([
        ("words", TfidfVectorizer(preprocessor=clean_text, ngram_range=(1, 2),
                                  token_pattern=r"(?u)\b\w+\b|[^\w\s]",
                                  min_df=3, max_df=0.9, max_features=100_000,
                                  sublinear_tf=True, dtype=np.float32)),
        ("chars", TfidfVectorizer(preprocessor=clean_text, analyzer="char_wb",
                                  ngram_range=(3, 5), min_df=5, max_features=150_000,
                                  sublinear_tf=True, dtype=np.float32)),
    ])),
    ("classifier", LogisticRegression(C=0.5, max_iter=2000, solver="liblinear", random_state=42)),
])

start = time.time()
old_style_model.fit(X_tr, y_tr)
print(f"Old-recipe model trained in {time.time() - start:.0f} s")

start = time.time()
improved_model.fit(X_tr_aug, y_tr_aug)
print(f"Improved model trained in {time.time() - start:.0f} s")
```

## Cell 63

Prepares the extra human text from blogs and BBC news, and tests the current model on it.

```
# Step 25: Extra human-written text from other domains
# Blog Authorship Corpus (blogs from 2004) and BBC News (2004-05) were written
# long before AI text generators existed, so they are reliably human-written.
blog_path = glob.glob('/kaggle/input/**/blogtext.csv', recursive=True)[0]
bbc_path = glob.glob('/kaggle/input/**/bbc_data.csv', recursive=True)[0]

def tidy(s):
    s = s.astype(str).str.replace("urlLink", " ", regex=False)   # blog corpus link marker
    return s.str.replace(r"\s+", " ", regex=True).str.strip()

blogs = tidy(pd.read_csv(blog_path, usecols=["text"], encoding_errors="ignore")["text"])
wc = blogs.str.split().str.len()
letters = blogs.str.count(r"[A-Za-z]") / blogs.str.len().clip(lower=1)
blogs = blogs[(wc >= 50) & (wc <= 800) & (letters > 0.6)].drop_duplicates()
blogs = blogs.sample(n=min(20_000, len(blogs)), random_state=42)

bbc = tidy(pd.read_csv(bbc_path)["data"]).drop_duplicates()

extra_human = pd.concat([blogs, bbc], ignore_index=True).sample(frac=1,
random_state=42).reset_index(drop=True)
n_test = int(0.20 * len(extra_human))
n_val = int(0.15 * len(extra_human))
```

```
extra_test = extra_human.iloc[:n_test]
extra_val = extra_human.iloc[n_test:n_test + n_val]
extra_train = extra_human.iloc[n_test + n_val:]
extra_test_short = extra_test.map(random_snippet)

print(f"Blog posts: {len(blogs):,} | BBC articles: {len(bbc):,}")
print(f"Extra human text -> train: {len(extra_train):,}, validation: {len(extra_val):,}, test:
{len(extra_test):,}")

# How does the CURRENT model handle this unseen, non-essay human writing?
print(f"\nCURRENT MODEL on unseen blog/news human text (AI threshold {ai_threshold:.2f})")
for name, X in [("Full posts/articles", extra_test), ("Short snippets", extra_test_short)]:
    p = improved_model.predict_proba(X)[:, 1]
    print(f"{name:20s} flagged as AI: {100 * (p >= ai_threshold).mean():5.2f}% | "
          f"Uncertain: {100 * ((p >= 0.5) & (p < ai_threshold)).mean():5.2f}%")
```

## Cell 64

Retrains as the final model (v2), chooses the threshold on validation data, compares results on held-out data and saves the files.

```
# Step 26: Retrain with the extra human text and compare with the current model
from sklearn.base import clone

extra_train_aug = pd.concat([extra_train, extra_train.map(random_snippet)], ignore_index=True)
X_tr_v2 = pd.concat([X_tr_aug, extra_train_aug], ignore_index=True)
y_tr_v2 = pd.concat([y_tr_aug, pd.Series(0, index=range(len(extra_train_aug)))], ignore_index=True)
print(f"Training rows: {len(X_tr_v2):,} (AI share {100 * y_tr_v2.mean():.1f}%)")

start = time.time()
model_v2 = clone(improved_model).fit(X_tr_v2, y_tr_v2)
print(f"Model v2 trained in {time.time() - start:.0f} s")

# Pick the threshold on validation data (original validation + extra human validation)
extra_val_aug = pd.concat([extra_val, extra_val.map(random_snippet)], ignore_index=True)
X_val_v2 = pd.concat([X_val_aug, extra_val_aug], ignore_index=True)
y_val_v2 = np.concatenate([y_val_aug.to_numpy(), np.zeros(len(extra_val_aug), dtype=int)])
pv = model_v2.predict_proba(X_val_v2)[:, 1]
cands = [t for t in np.round(np.arange(0.50, 0.96, 0.05), 2) if (pv[y_val_v2 == 1] >= t).mean() >= 0.90]
threshold_v2 = float(max(cands)) if cands else 0.5
print(f"Model v2 AI threshold: {threshold_v2:.2f}")

# Compare on held-out test data (never used for training or threshold choice)
yt = y_te.to_numpy()
rows = []
for model_name, m, thr in [("Current model", improved_model, ai_threshold), ("Model v2 (+blogs/news)",
model_v2, threshold_v2)]:
    for test_name, X, y in [("Essays - full", X_te, yt), ("Essays - short", X_te_short, yt),
                            ("Blogs/news (human) - full", extra_test, np.zeros(len(extra_test), int)),
                            ("Blogs/news (human) - short", extra_test_short, np.zeros(len(extra_test),
int))]:
        p = m.predict_proba(X)[:, 1]
        rows.append({"Model": model_name, "Test set": test_name,
                     "Human flagged as AI (%)": 100 * (p[y == 0] >= thr).mean(),
                     "Human Uncertain (%)": 100 * ((p[y == 0] >= 0.5) & (p[y == 0] < thr)).mean(),
                     "AI caught (%)": 100 * (p[y == 1] >= thr).mean() if (y == 1).any() else np.nan})
print("\nHELD-OUT TEST COMPARISON")
print(pd.DataFrame(rows).round(2).to_string(index=False))

# Use model v2 from now on and save it
improved_model, ai_threshold = model_v2, threshold_v2
joblib.dump(improved_model, os.path.join(model_dir, "improved_lr_pipeline.joblib"))
json.dump({"ai_threshold": ai_threshold, "min_words": MIN_WORDS},
          open(os.path.join(model_dir, "improved_model_config.json"), "w"))
print("\nSaved model v2 + threshold. Re-run Step 24 to test your own texts.")
```

# 5. saved models: The Trained Model

These files connect the notebook to the app. They are created by the notebook and only read by the backend.

### `saved models/improved_lr_pipeline.joblib`

9.6 MB

The final trained model, saved by joblib as a binary file of about 9.6 MB. It is a scikit-learn `Pipeline`: a `FeatureUnion` of a word TF-IDF (100,000 features) and a character TF-IDF (150,000 features), followed by `LogisticRegression(C=0.5)`. It holds learned vocabularies and weights rather than readable code. It needs scikit-learn 1.6.1 and `clean_text.py` to load.

### `saved models/clean_text.py`

215 bytes · 7 lines

Defines `clean_text()`, which lower-cases text, turns curly quotes into straight ones and collapses repeated whitespace. Both TF-IDF steps call it, so text is treated the same way in training and prediction. The notebook saved the model with a reference to this function, so **the model cannot load without it**.

```python
import re

def clean_text(text):
    text = str(text).lower()
    for a, b in {"’": "'", "‘": "'", "“": '"', "”": '"'}.items():
        text = text.replace(a, b)
    return re.sub(r"\s+", " ", text).strip()
```

### `saved models/improved_model_config.json`

39 bytes · 1 lines

Stores two decision settings beside the model. `ai_threshold` is 0.65: text is labelled AI only at or above this probability. `min_words` is 50: the app rejects anything shorter.

```json
{"ai_threshold": 0.65, "min_words": 50}
```

### `saved models/ai_text_detector_model.zip`

4.2 MB

A zip downloaded from Kaggle containing the joblib model, `clean_text.py` and the config file. The app does not read it. It is a convenient copy for moving the model.

# 6. backend: The API Server

**main.py**
Receives requests and sends responses

→

**model_service.py**
Loads the model and predicts

→

**config.py**
Supplies paths and settings from .env

## backend/requirements.txt

297 bytes · 12 lines

Lists the Python packages for `pip install -r`. `scikit-learn` is pinned to exactly 1.6.1 to match the version that saved the model.

```
# scikit-learn MUST match the version the model was trained/saved with on Kaggle (1.6.1),
# otherwise joblib may fail to load the pipeline or behave differently.
scikit-learn==1.6.1
joblib>=1.3
numpy>=1.26
fastapi>=0.110
uvicorn[standard]>=0.29
python-dotenv>=1.0

# Tests
pytest>=8.0
httpx>=0.27
```

## backend/.env.example

1.0 KB · 24 lines

A commented template for the settings file. Copy it to `.env` and fill in the values. This is safe to share because it contains no machine-specific paths.

```
# Copy this file to backend/.env and edit the values.
# Paths can be absolute, or relative to the backend/ folder.

# REQUIRED: the trained pipeline saved from your Kaggle notebook.
#   MODEL_PATH=../saved models/improved_lr_pipeline.joblib
MODEL_PATH=

# REQUIRED for improved_lr_pipeline.joblib: the clean_text() function the pipeline was trained with.
#   CLEAN_TEXT_PATH=../saved models/clean_text.py
CLEAN_TEXT_PATH=

# Decision threshold and minimum word count saved with the model ({"ai_threshold": ..., "min_words": ...}).
# If left empty, the label is whichever probability is higher and the minimum is 3 words.
#   MODEL_CONFIG_PATH=../saved models/improved_model_config.json
MODEL_CONFIG_PATH=

# Class value the model uses for AI-generated text (dataset column `generated`: 1 = AI, 0 = human).
AI_CLASS_LABEL=1

# Comma-separated list of frontend origins allowed by CORS (Vite dev server by default).
ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173

# Maximum input length in characters.
MAX_TEXT_CHARS=50000
```

## backend/.env

239 bytes · 5 lines

The settings the backend really uses on this machine: where the three model files are, which class means AI and which frontend address is allowed. Relative paths start from the `backend` folder. Git ignores this file.

```
MODEL_PATH=../saved models/improved_lr_pipeline.joblib
CLEAN_TEXT_PATH=../saved models/clean_text.py
MODEL_CONFIG_PATH=../saved models/improved_model_config.json
```

```
AI_CLASS_LABEL=1
ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

## backend/models/README.txt

189 bytes · 2 lines

An optional alternative location for the model files. The folder contains only this note, because the current settings point to `saved models/`.

```
Optional: place improved_lr_pipeline.joblib, clean_text.py and improved_model_config.json here
and point MODEL_PATH, CLEAN_TEXT_PATH and MODEL_CONFIG_PATH in backend/.env to models/<file>.
```

## backend/app/__init__.py

0 bytes

An empty file. Its presence makes `app` a Python package, which allows imports such as `from .config import settings` and the server command `app.main:app`.

## backend/app/config.py

1.9 KB · 49 lines

Loads `backend/.env` and puts all settings into one `settings` object for the other files to use.

- `_resolve_path()` turns a relative path into a full path, starting from the backend folder rather than wherever the command was run.
- `_split_csv()` splits the comma-separated list of allowed origins.
- `Settings` is a frozen dataclass holding the model paths, AI class label, allowed origins and maximum text length.
- No model path has a default, so the path is never guessed.

```python
"""Application settings, read from environment variables (or backend/.env)."""

import os
from dataclasses import dataclass, field
from pathlib import Path

from dotenv import load_dotenv

BACKEND_DIR = Path(__file__).resolve().parent.parent

# Load backend/.env if present. Real environment variables take precedence.
load_dotenv(BACKEND_DIR / ".env")


def _resolve_path(raw: str | None) -> Path | None:
    """Relative paths are resolved against the backend folder, not the shell's cwd."""
    if not raw or not raw.strip():
        return None
    path = Path(raw.strip().strip('"').strip("'")).expanduser()
    if not path.is_absolute():
        path = BACKEND_DIR / path
    return path.resolve()


def _split_csv(raw: str) -> list[str]:
    return [item.strip() for item in raw.split(",") if item.strip()]


@dataclass(frozen=True)
class Settings:
    # Path to the trained scikit-learn pipeline (.joblib). Required - no default is guessed.
    model_path: Path | None = field(default_factory=lambda: _resolve_path(os.getenv("MODEL_PATH")))
    # Python file defining clean_text(), which the pipeline's TF-IDF vectorizers use as preprocessor.
    clean_text_path: Path | None = field(default_factory=lambda:
_resolve_path(os.getenv("CLEAN_TEXT_PATH")))
    # JSON with {"ai_threshold": ..., "min_words": ...} saved alongside the model.
```

```
    model_config_path: Path | None = field(
        default_factory=lambda: _resolve_path(os.getenv("MODEL_CONFIG_PATH"))
    )
    # The class value the model uses for AI-generated text (the dataset's `generated` column: 1 = AI).
    ai_class_label: str = field(default_factory=lambda: os.getenv("AI_CLASS_LABEL", "1").strip())
    allowed_origins: list[str] = field(
        default_factory=lambda: _split_csv(
            os.getenv("ALLOWED_ORIGINS", "http://localhost:5173,http://127.0.0.1:5173")
        )
    )
    max_text_chars: int = field(default_factory=lambda: int(os.getenv("MAX_TEXT_CHARS", "50000")))


settings = Settings()
```

## backend/app/model_service.py

6.8 KB · 174 lines

The bridge between the API and the trained model. It handles everything to do with loading the model, checking it and getting predictions from it.

- `ModelService.load()` checks the paths, reads the config, registers `clean_text`, loads the joblib file and confirms it is a two-class model with `predict_proba()`. Any failure is stored as a readable message instead of crashing the server.
- `_register_clean_text()` imports your `clean_text.py` and places the function on `__main__`, where the saved model expects to find it.
- `_load_model_config()` reads and validates the threshold and minimum words.
- `predict()` calls `predict_proba()` and applies the rule: AI-Generated if the AI probability is at least the threshold, otherwise Human-Written.
- `describe()` reports the pipeline's structure for the health endpoint.

```
"""Loads the trained Logistic Regression pipeline and turns predict_proba() output into a response."""

import __main__
import importlib.util
import json
import logging
from dataclasses import dataclass
from pathlib import Path

import joblib

logger = logging.getLogger(__name__)

# Used only when no model config JSON is given: plain argmax and a minimal word count.
DEFAULT_AI_THRESHOLD = 0.5
DEFAULT_MIN_WORDS = 3


class ModelNotReadyError(RuntimeError):
    """Raised when a prediction is requested but the model could not be loaded."""


@dataclass
class Prediction:
    label: str  # "AI-Generated" or "Human-Written"
    human_probability: float
    ai_probability: float


class ModelService:
    def __init__(
        self,
        model_path: Path | None,
        ai_class_label: str,
```

```
        clean_text_path: Path | None = None,
        model_config_path: Path | None = None,
    ):
        self.model_path = model_path
        self.ai_class_label = ai_class_label
        self.clean_text_path = clean_text_path
        self.model_config_path = model_config_path
        self.ai_threshold = DEFAULT_AI_THRESHOLD
        self.min_words = DEFAULT_MIN_WORDS
        self.model = None
        self.error: str | None = None
        self._ai_index: int | None = None
        self._human_index: int | None = None

    @property
    def is_ready(self) -> bool:
        return self.model is not None

    def _fail(self, message: str) -> None:
        self.error = message
        logger.error(message)

    def _register_clean_text(self) -> bool:
        """The notebook pickled the pipeline with `__main__.clean_text` as the TF-IDF preprocessor,
        so that function must exist on __main__ before joblib can unpickle the model."""
        if self.clean_text_path is None:
            return True
        if not self.clean_text_path.is_file():
            self._fail(f"clean_text.py not found at '{self.clean_text_path}'. Update CLEAN_TEXT_PATH.")
            return False
        try:
            spec = importlib.util.spec_from_file_location("clean_text", self.clean_text_path)
            module = importlib.util.module_from_spec(spec)
            spec.loader.exec_module(module)
            __main__.clean_text = module.clean_text
        except Exception as exc:
            self._fail(f"Could not import clean_text from '{self.clean_text_path}': {exc}")
            return False
        return True

    def _load_model_config(self) -> bool:
        if self.model_config_path is None:
            return True
        if not self.model_config_path.is_file():
            self._fail(f"Model config not found at '{self.model_config_path}'. Update MODEL_CONFIG_PATH.")
            return False
        try:
            config = json.loads(self.model_config_path.read_text(encoding="utf-8"))
            self.ai_threshold = float(config.get("ai_threshold", DEFAULT_AI_THRESHOLD))
            self.min_words = int(config.get("min_words", DEFAULT_MIN_WORDS))
        except (ValueError, TypeError, AttributeError) as exc:
            self._fail(f"Invalid model config '{self.model_config_path}': {exc}")
            return False
        if not 0 < self.ai_threshold < 1:
            self._fail(f"ai_threshold must be between 0 and 1, got {self.ai_threshold}.")
            return False
        return True

    def load(self) -> None:
        """Load the model once at startup. On failure, store a setup message instead of raising."""
        if self.model_path is None:
            self._fail(
                "MODEL_PATH is not set. Copy backend/.env.example to backend/.env and set MODEL_PATH "
                "to your improved_lr_pipeline.joblib file."
            )
            return

        if not self.model_path.is_file():
            self._fail(
                f"Model file not found at '{self.model_path}'. Copy improved_lr_pipeline.joblib from "
                "Kaggle and update MODEL_PATH."
```

```
            )
            return

        if not self._load_model_config() or not self._register_clean_text():
            return

        try:
            model = joblib.load(self.model_path)
        except AttributeError as exc:
            if "clean_text" in str(exc):
                self._fail(
                    "The model needs the clean_text() function it was trained with. "
                    "Set CLEAN_TEXT_PATH to your clean_text.py file."
                )
            else:
                self._fail(f"Could not load model from '{self.model_path}': {exc}")
            return
        except Exception as exc:  # unpickling errors vary widely (version mismatch, corrupt file, ...)
            logger.exception("Model load failed")
            self._fail(
                f"Could not load model from '{self.model_path}': {type(exc).__name__}: {exc}. "
                "Check that scikit-learn matches the version used on Kaggle (see requirements.txt)."
            )
            return

        if not hasattr(model, "predict_proba") or not hasattr(model, "classes_"):
            self._fail("The loaded object is not a fitted classifier with predict_proba().")
            return

        classes = [str(c) for c in model.classes_]
        if len(classes) != 2 or self.ai_class_label not in classes:
            self._fail(
                f"Model classes are {classes}, but AI_CLASS_LABEL is '{self.ai_class_label}'. "
                "Set AI_CLASS_LABEL to the class value that means AI-generated."
            )
            return

        self._ai_index = classes.index(self.ai_class_label)
        self._human_index = 1 - self._ai_index
        self.model = model
        self.error = None
        logger.info(
            "Loaded model from %s (classes=%s, ai_threshold=%s, min_words=%s)",
            self.model_path, classes, self.ai_threshold, self.min_words,
        )

    def predict(self, text: str) -> Prediction:
        if not self.is_ready:
            raise ModelNotReadyError(self.error or "Model is not loaded.")

        probabilities = self.model.predict_proba([text])[0]
        ai_probability = float(probabilities[self._ai_index])
        human_probability = float(probabilities[self._human_index])
        # The tuned threshold from the notebook: text is only called AI when the model is confident enough.
        label = "AI-Generated" if ai_probability >= self.ai_threshold else "Human-Written"
        return Prediction(label, human_probability, ai_probability)

    def describe(self) -> dict:
        """Pipeline structure, for the /health endpoint."""
        if not self.is_ready:
            return {}
        described = {}
        for name, step in getattr(self.model, "named_steps", {}).items():
            parts = getattr(step, "transformer_list", None)
            if parts:
                described[name] = {sub: type(t).__name__ for sub, t in parts}
            else:
                described[name] = type(step).__name__
        return described
```

## backend/app/main.py

4.4 KB · 123 lines

Creates the web server and defines the REST API that the frontend calls. It handles requests and responses, and leaves the ML work to `model_service.py`.

- `lifespan()` loads the model once when the server starts.
- `CORSMiddleware` allows the browser frontend, which runs on a different port, to call the API.
- `PredictRequest.validate_text()` trims the text and rejects empty or over-long input.
- `validation_error_handler()` turns validation errors into one simple message.
- `GET /health` reports whether the model loaded, its structure and limits, and any setup error.
- `POST /predict` enforces the minimum word count, calls the model and returns the label and probabilities. It returns HTTP 422 for bad input, 503 if the model is not loaded and 500 for unexpected errors.

```python
"""FastAPI app: GET /health and POST /predict for AI vs Human text classification."""

import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, HTTPException, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field, field_validator

from .config import settings
from .model_service import ModelNotReadyError, ModelService

logging.basicConfig(level=logging.INFO, format="%(levelname)s  %(name)s: %(message)s")

DISCLAIMER = "These are machine-learning estimates, not proof of authorship."

model_service = ModelService(
    settings.model_path,
    settings.ai_class_label,
    clean_text_path=settings.clean_text_path,
    model_config_path=settings.model_config_path,
)


@asynccontextmanager
async def lifespan(_: FastAPI):
    model_service.load()
    yield


app = FastAPI(
    title="AI vs Human Text Classifier API",
    description="Serves a trained word + character TF-IDF + Logistic Regression pipeline.",
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.allowed_origins,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)


class PredictRequest(BaseModel):
    text: str = Field(..., description="The text to classify.")

    @field_validator("text")
    @classmethod
    def validate_text(cls, value: str) -> str:
```

```
        stripped = value.strip()
        if not stripped:
            raise ValueError("Text is empty. Please paste some text to analyze.")
        if len(stripped) > settings.max_text_chars:
            raise ValueError(
                f"Text is too long ({len(stripped):,} characters). "
                f"The limit is {settings.max_text_chars:,} characters."
            )
        return stripped


class PredictResponse(BaseModel):
    label: str = Field(..., description="'AI-Generated' or 'Human-Written'")
    human_probability: float = Field(..., ge=0, le=1)
    ai_probability: float = Field(..., ge=0, le=1)
    ai_threshold: float = Field(..., description="AI probability at or above which the label is AI-Generated")
    disclaimer: str = DISCLAIMER


@app.exception_handler(RequestValidationError)
async def validation_error_handler(_: Request, exc: RequestValidationError):
    # Flatten pydantic's error list into one readable message for the frontend.
    messages = []
    for error in exc.errors():
        msg = str(error.get("msg", "Invalid input"))
        messages.append(msg.removeprefix("Value error, "))
    return JSONResponse(status_code=422, content={"detail": " ".join(messages) or "Invalid request."})


@app.get("/health")
def health():
    return {
        "status": "ok" if model_service.is_ready else "model_not_loaded",
        "model_loaded": model_service.is_ready,
        "model_path": str(model_service.model_path) if model_service.model_path else None,
        "pipeline_steps": model_service.describe(),
        "ai_threshold": model_service.ai_threshold,
        "min_words": model_service.min_words,
        "max_chars": settings.max_text_chars,
        "error": model_service.error,
    }


@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    # The minimum comes from the model config, so it is checked here rather than in the schema.
    word_count = len(request.text.split())
    if model_service.is_ready and word_count < model_service.min_words:
        raise HTTPException(
            status_code=422,
            detail=(
                f"Please enter at least {model_service.min_words} words (you entered {word_count}). "
                "The model is unreliable on very short texts."
            ),
        )

    try:
        result = model_service.predict(request.text)
    except ModelNotReadyError as exc:
        raise HTTPException(status_code=503, detail=f"Model not available: {exc}") from exc
    except Exception as exc:
        logging.getLogger(__name__).exception("Prediction failed")
        raise HTTPException(status_code=500, detail="Prediction failed due to an internal error.") from exc

    return PredictResponse(
        label=result.label,
        human_probability=result.human_probability,
        ai_probability=result.ai_probability,
        ai_threshold=model_service.ai_threshold,
    )
```

## backend/tests/test_api.py

4.3 KB · 119 lines

Eleven pytest tests, run with `python -m pytest -q` from the backend folder.

- `test_predict_matches_pipeline_predict_proba` proves the API returns exactly what the real model produces, and that the label follows the threshold.
- Other tests cover empty text, missing fields, too few words and how the threshold decides a label.
- The setup tests confirm that a missing model, unset path or missing `clean_text.py` gives a clear error.

```python
"""API tests. Prediction tests run against the real model and are skipped if MODEL_PATH is not usable."""

import json

import pytest
from fastapi.testclient import TestClient

from app.config import settings
from app.main import app, model_service
from app.model_service import ModelService

model_available = settings.model_path is not None and settings.model_path.is_file()
needs_model = pytest.mark.skipif(not model_available, reason="MODEL_PATH not configured")

LONG_TEXT = (
    "Technology has changed the way students learn in many important ways. Online resources make it "
    "easier to research topics, collaborate with classmates, and get feedback from teachers. However, "
    "students also face more distractions than ever before, and they need to develop good habits to "
    "manage their time. In conclusion, technology offers clear benefits for education, but only when "
    "it is used thoughtfully and responsibly by both students and teachers in the classroom."
)


@pytest.fixture(scope="module")
def client():
    with TestClient(app) as c:  # runs the lifespan, which loads the model
        yield c


def test_health(client):
    body = client.get("/health").json()
    assert body["model_loaded"] is model_available, body["error"]
    if model_available:
        assert body["pipeline_steps"]["classifier"] == "LogisticRegression"
        assert 0 < body["ai_threshold"] < 1


@needs_model
def test_predict_matches_pipeline_predict_proba(client):
    res = client.post("/predict", json={"text": LONG_TEXT})
    assert res.status_code == 200, res.text
    body = res.json()
    assert body["human_probability"] + body["ai_probability"] == pytest.approx(1.0)

    # The API must return exactly what the trained pipeline produces.
    classes = [str(c) for c in model_service.model.classes_]
    expected = model_service.model.predict_proba([LONG_TEXT])[0]
    assert body["ai_probability"] == pytest.approx(expected[classes.index(settings.ai_class_label)])

    # And the label must follow the configured threshold.
    expected_label = "AI-Generated" if body["ai_probability"] >= body["ai_threshold"] else "Human-Written"
    assert body["label"] == expected_label


@pytest.mark.parametrize("text", ["", "   \n\t "])
def test_predict_rejects_empty_text(client, text):
    res = client.post("/predict", json={"text": text})
    assert res.status_code == 422
    assert "empty" in res.json()["detail"].lower()
```

```python


def test_predict_rejects_missing_field(client):
    assert client.post("/predict", json={}).status_code == 422


@needs_model
def test_predict_rejects_too_few_words(client):
    res = client.post("/predict", json={"text": "just a few words here"})
    assert res.status_code == 422
    assert f"at least {model_service.min_words} words" in res.json()["detail"]


def test_threshold_decides_label(tmp_path):
    service = ModelService(None, "1")
    service.ai_threshold = 0.65
    service._ai_index, service._human_index = 1, 0

    class Fake:
        def __init__(self, p_ai):
            self.p_ai = p_ai

        def predict_proba(self, _):
            return [[1 - self.p_ai, self.p_ai]]

    service.model = Fake(0.60)
    assert service.predict("x").label == "Human-Written"  # higher than human, but below threshold
    service.model = Fake(0.70)
    assert service.predict("x").label == "AI-Generated"


def test_missing_model_file_gives_setup_error(tmp_path):
    service = ModelService(tmp_path / "does_not_exist.joblib", "1")
    service.load()
    assert not service.is_ready
    assert "not found" in service.error


def test_unset_model_path_gives_setup_error():
    service = ModelService(None, "1")
    service.load()
    assert not service.is_ready
    assert "MODEL_PATH is not set" in service.error


def test_missing_clean_text_file_gives_setup_error(tmp_path):
    model = tmp_path / "m.joblib"
    model.write_bytes(b"")
    service = ModelService(model, "1", clean_text_path=tmp_path / "clean_text.py")
    service.load()
    assert not service.is_ready
    assert "clean_text.py not found" in service.error


def test_model_config_is_read(tmp_path):
    config = tmp_path / "cfg.json"
    config.write_text(json.dumps({"ai_threshold": 0.7, "min_words": 20}))
    service = ModelService(None, "1", model_config_path=config)
    assert service._load_model_config()
    assert (service.ai_threshold, service.min_words) == (0.7, 20)
```

# 7. frontend: The Web Interface

**main.jsx**
Starts the app

→

**App.jsx**
Holds the state and builds the page

→

**components/**
Pieces of the page

→

**lib/api.js**
Calls the backend

## `frontend/index.html`

583 bytes · 15 lines

The single HTML page of the app. It sets the title, icon and Inter font, provides the empty `<div id="root">` and loads `src/main.jsx`.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>AI vs Human Text Classifier</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet" />
  </head>
  <body class="bg-navy-950">
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

## `frontend/package.json`

443 bytes · 22 lines

Lists the frontend's packages with exact versions, and defines the commands `npm run dev`, `build` and `preview`.

```
{
  "name": "ai-human-text-classifier",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "18.3.1",
    "react-dom": "18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "4.3.4",
    "autoprefixer": "10.4.20",
    "postcss": "8.4.49",
    "tailwindcss": "3.4.17",
    "vite": "5.4.11"
  }
}
```

### `frontend/package-lock.json`

86.4 KB

Generated automatically by npm. It records the exact version of every package and sub-package so installs are repeatable. It is about 88 KB and should never be edited by hand, so it is not printed here.

### `frontend/vite.config.js`

180 bytes · 7 lines

Configures Vite, the frontend's development server and build tool. It enables the React plugin and fixes the port at 5173.

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: { port: 5173, strictPort: true },
});
```

### `frontend/tailwind.config.js`

378 bytes · 15 lines

Configures Tailwind CSS: which files to scan for class names, the Inter font and the custom `navy` colours behind the dark dashboard.

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,jsx}"],
  theme: {
    extend: {
      fontFamily: {
        sans: ["Inter", "system-ui", "Segoe UI", "Roboto", "sans-serif"],
      },
      colors: {
        navy: { 950: "#060b1a", 900: "#0a1128", 800: "#111a3a", 700: "#1a2550" },
      },
    },
  },
  plugins: [],
};
```

### `frontend/postcss.config.js`

70 bytes · 3 lines

Tells PostCSS to run Tailwind and Autoprefixer over the CSS during the build.

```
export default {
  plugins: { tailwindcss: {}, autoprefixer: {} },
};
```

### `frontend/.env.example`

101 bytes · 2 lines

An optional template. Copy it to `frontend/.env` only if the backend is not at `http://localhost:8000`.

```
# URL of the FastAPI backend. Copy to frontend/.env to change it.
VITE_API_URL=http://localhost:8000
```

## frontend/public/favicon.svg

373 bytes · 1 lines

The small purple icon shown on the browser tab, drawn as an SVG.

```svg
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 32 32"><defs><linearGradient id="g" x1="0" y1="0" x2="1" y2="1"><stop offset="0" stop-color="#818cf8"/><stop offset="1" stop-color="#c084fc"/></linearGradient></defs><rect width="32" height="32" rx="8" fill="url(#g)"/><path d="M9 11h14M9 16h10M9 21h12" stroke="#0a1128" stroke-width="2.5" stroke-linecap="round"/></svg>
```

## frontend/src/main.jsx

239 bytes · 10 lines

The entry point. It imports the styles and renders `<App />` into the `root` element of `index.html`.

```jsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

## frontend/src/index.css

819 bytes · 25 lines

Loads Tailwind and defines the reusable styles used across the app: `.card`, `.btn`, `.btn-primary` and `.btn-ghost`.

```css
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  body {
    @apply bg-navy-950 text-slate-200 font-sans antialiased;
  }
}

@layer components {
  .card {
    @apply rounded-2xl border border-white/10 bg-navy-900/70 p-5 shadow-xl shadow-black/20 backdrop-blur sm:p-6;
  }
  .btn {
    @apply inline-flex items-center justify-center gap-2 rounded-xl px-4 py-2.5 text-sm font-medium transition
      focus:outline-none focus-visible:ring-2 focus-visible:ring-indigo-400 disabled:cursor-not-allowed disabled:opacity-50;
  }
  .btn-primary {
    @apply btn bg-gradient-to-r from-indigo-500 to-violet-500 text-white shadow-lg shadow-indigo-500/25 hover:from-indigo-400 hover:to-violet-400;
  }
  .btn-ghost {
    @apply btn border border-white/10 bg-white/5 text-slate-200 hover:bg-white/10;
  }
}
```

## frontend/src/App.jsx

4.2 KB · 115 lines

The top-level component. It holds the app's state, connects all the components and lays out the page.

- State: the text, result, error, loading flag and backend health.
- `checkHealth()` calls `/health` when the page opens and when Retry is pressed.
- `handleAnalyze()` sends the text, cancels any request still running, shows the result or error, and adds successful results to the history.
- `handleClear()` and `handleSelectHistory()` reset the page or show an earlier result.
- The limits `maxChars` and `minWords` come from the backend's `/health` response, so there is one source of truth.

```
import { useCallback, useEffect, useRef, useState } from "react";
import AboutModel from "./components/AboutModel";
import Header from "./components/Header";
import HistoryPanel from "./components/HistoryPanel";
import ResultCard from "./components/ResultCard";
import SetupBanner from "./components/SetupBanner";
import TextInput from "./components/TextInput";
import { getHealth, predictText } from "./lib/api";
import { useSessionHistory } from "./lib/useSessionHistory";

// Fallbacks until /health reports the backend's actual limits.
const DEFAULT_MAX_CHARS = 50000;
const DEFAULT_MIN_WORDS = 1;

export default function App() {
  const [text, setText] = useState("");
  const [result, setResult] = useState(null);
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [health, setHealth] = useState({ loading: true, data: null, error: null });
  const { history, add, clear } = useSessionHistory();
  const abortRef = useRef(null);
  const maxChars = health.data?.max_chars ?? DEFAULT_MAX_CHARS;
  const minWords = health.data?.model_loaded ? health.data.min_words : DEFAULT_MIN_WORDS;

  const checkHealth = useCallback(async () => {
    setHealth((h) => ({ ...h, loading: true }));
    try {
      setHealth({ loading: false, data: await getHealth(), error: null });
    } catch (e) {
      setHealth({ loading: false, data: null, error: e.message });
    }
  }, []);

  useEffect(() => {
    checkHealth();
  }, [checkHealth]);

  const handleAnalyze = async () => {
    if (!text.trim()) {
      setError("Please paste some text before analyzing.");
      return;
    }
    abortRef.current?.abort();
    const controller = new AbortController();
    abortRef.current = controller;

    setLoading(true);
    setError("");
    try {
      const data = await predictText(text, controller.signal);
      setResult(data);
      add({ id: `${Date.now()}-${Math.random()}`, time: Date.now(), text: text.trim().slice(0, 300),
...data });
    } catch (e) {
      if (e.name === "AbortError") return;
```

```
      setResult(null);
      setError(e.message);
      // The backend may have gone down or failed to load the model - refresh the status banner.
      if (e.message.startsWith("Cannot reach") || e.message.startsWith("Model not available"))
checkHealth();
    } finally {
      if (abortRef.current === controller) setLoading(false);
    }
  };

  const handleClear = () => {
    setText("");
    setResult(null);
    setError("");
  };

  const handleSelectHistory = (item) => {
    setResult(item);
    setError("");
    window.scrollTo({ top: 0, behavior: "smooth" });
  };

  return (
    <div className="relative min-h-screen overflow-x-hidden">
      <div aria-hidden="true" className="pointer-events-none absolute inset-0 -z-10 overflow-hidden">
        <div className="absolute -left-40 -top-40 h-[28rem] w-[28rem] rounded-full bg-indigo-600/20 blur-
3xl" />
        <div className="absolute -right-32 top-24 h-[24rem] w-[24rem] rounded-full bg-violet-600/20 blur-
3xl" />
        <div className="absolute bottom-0 left-1/3 h-[20rem] w-[20rem] rounded-full bg-blue-700/10 blur-
3xl" />
      </div>

      <main className="mx-auto max-w-6xl space-y-6 px-4 py-8 sm:px-6 sm:py-12">
        <Header health={health} />
        <SetupBanner health={health} onRetry={checkHealth} />

        <div className="grid gap-6 lg:grid-cols-5">
          <div className="lg:col-span-3">
            <TextInput
              text={text}
              onChange={setText}
              onAnalyze={handleAnalyze}
              onClear={handleClear}
              loading={loading}
              maxChars={maxChars}
              minWords={minWords}
            />
          </div>
          <div className="lg:col-span-2">
            <ResultCard result={result} error={error} />
          </div>
        </div>

        <HistoryPanel history={history} onSelect={handleSelectHistory} onClear={clear} />
        <AboutModel health={health} />

        <footer className="pb-4 text-center text-xs text-slate-500">
          College ML project · TF-IDF + Logistic Regression · Results are estimates, not proof of
authorship.
        </footer>
      </main>
    </div>
  );
}
```

## frontend/src/components/Header.jsx

1.3 KB · 29 lines

Displays the page title and a small status badge: Checking backend, Model loaded, Model not loaded or Backend offline.

```jsx
export default function Header({ health }) {
  const status = health.loading
    ? { text: "Checking backend…", dot: "bg-slate-400 animate-pulse" }
    : health.error
      ? { text: "Backend offline", dot: "bg-rose-400" }
      : health.data?.model_loaded
        ? { text: "Model loaded", dot: "bg-emerald-400" }
        : { text: "Model not loaded", dot: "bg-amber-400" };

  return (
    <header className="flex flex-col gap-4 sm:flex-row sm:items-end sm:justify-between">
      <div>
        <p className="text-xs font-semibold uppercase tracking-[0.2em] text-indigo-300/80">
          Machine Learning Project
        </p>
        <h1 className="mt-2 bg-gradient-to-r from-white via-indigo-100 to-violet-300 bg-clip-text text-3xl
font-bold text-transparent sm:text-4xl">
          AI vs Human Text Classifier
        </h1>
        <p className="mt-2 max-w-2xl text-sm text-slate-400">
          Paste a passage to see how a word + character TF-IDF and Logistic Regression model classifies it.
        </p>
      </div>
      <div className="inline-flex items-center gap-2 self-start rounded-full border border-white/10 bg-
white/5 px-3 py-1.5 text-xs text-slate-300 sm:self-auto">
        <span className={`h-2 w-2 rounded-full ${status.dot}`} />
        {status.text}
      </div>
    </header>
  );
}
```

## frontend/src/components/SetupBanner.jsx

1.2 KB · 28 lines

An amber warning shown only when the backend is unreachable or the model failed to load. It displays the backend's own message and has a Retry button.

```jsx
import { API_URL } from "../lib/api";

/** Shown when the backend is unreachable or the model file failed to load. */
export default function SetupBanner({ health, onRetry }) {
  if (health.loading) return null;
  if (!health.error && health.data?.model_loaded) return null;

  const message = health.error
    ? `Cannot reach the backend at ${API_URL}. Start it from the backend folder with: uvicorn app.main:app -
-reload --port 8000`
    : health.data?.error || "The model is not loaded.";

  return (
    <div role="alert" className="rounded-2xl border border-amber-400/30 bg-amber-400/10 p-4 text-sm text-
amber-100">
      <div className="flex flex-col gap-3 sm:flex-row sm:items-start sm:justify-between">
        <div className="min-w-0">
          <p className="font-semibold text-amber-200">Setup required</p>
          <p className="mt-1 break-words text-amber-100/90">{message}</p>
          <p className="mt-1 text-amber-100/70">
            See the README section “Model file” for where to put the .joblib file.
          </p>
        </div>
        <button type="button" onClick={onRetry} className="btn-ghost shrink-0">
          Retry
```

```
        </button>
      </div>
    </div>
  );
}
```

## `frontend/src/components/TextInput.jsx`

3.4 KB · 83 lines

The input card: the large text area, character and word counts, the two sample buttons, Clear Text and Analyze Text.

- `canAnalyze` enables the button only when the text has enough words, is not too long and nothing is loading.
- Shows a warning when the text is too short or too long, and a spinner while analysing.
- Ctrl+Enter also starts the analysis.

```
import { SAMPLES } from "../lib/samples";
import { countWords } from "../lib/text";

export default function TextInput({ text, onChange, onAnalyze, onClear, loading, maxChars, minWords }) {
  const words = countWords(text);
  const chars = text.length;
  const tooLong = chars > maxChars;
  const tooShort = words > 0 && words < minWords;
  const canAnalyze = words >= Math.max(minWords, 1) && !tooLong && !loading;

  const handleKeyDown = (e) => {
    if ((e.ctrlKey || e.metaKey) && e.key === "Enter" && canAnalyze) onAnalyze();
  };

  return (
    <section className="card" aria-labelledby="input-heading">
      <div className="flex flex-wrap items-center justify-between gap-3">
        <h2 id="input-heading" className="text-lg font-semibold text-white">
          Text to analyze
        </h2>
        <div className="flex flex-wrap gap-2">
          {SAMPLES.map((s) => (
            <button
              key={s.id}
              type="button"
              className="btn-ghost px-3 py-1.5 text-xs"
              onClick={() => onChange(s.text)}
              disabled={loading}
              title="Example text from the project dataset"
            >
              {s.name}
            </button>
          ))}
        </div>
      </div>

      <label htmlFor="text-input" className="sr-only">
        Text to analyze
      </label>
      <textarea
        id="text-input"
        value={text}
        onChange={(e) => onChange(e.target.value)}
        onKeyDown={handleKeyDown}
        placeholder="Paste an essay, paragraph or article here…"
        className="mt-4 h-64 w-full resize-y rounded-xl border border-white/10 bg-navy-950/70 p-4 text-
[15px] leading-relaxed text-slate-100 placeholder:text-slate-500 focus:border-indigo-400/60 focus:outline-none
focus:ring-2 focus:ring-indigo-500/30 sm:h-72"
      />

      <div className="mt-3 flex flex-col gap-4 sm:flex-row sm:items-center sm:justify-between">
        <p className={`text-xs ${tooLong ? "text-rose-300" : tooShort ? "text-amber-300" : "text-slate-
400"}`}>
          <span className="font-medium text-slate-200">{chars.toLocaleString()}</span> characters ·{" "}
```

```
          <span className="font-medium text-slate-200">{words.toLocaleString()}</span> words
          {tooLong && ` · limit is ${maxChars.toLocaleString()} characters`}
          {tooShort && ` · at least ${minWords} words needed for a reliable estimate`}
        </p>
        <div className="flex gap-2">
          <button
            type="button"
            className="btn-ghost flex-1 sm:flex-none"
            onClick={onClear}
            disabled={loading || !text}
          >
            Clear Text
          </button>
          <button
            type="button"
            className="btn-primary flex-1 sm:flex-none"
            onClick={onAnalyze}
            disabled={!canAnalyze}
          >
            {loading && (
              <svg className="h-4 w-4 animate-spin" viewBox="0 0 24 24" fill="none" aria-hidden="true">
                <circle cx="12" cy="12" r="10" stroke="currentColor" strokeOpacity="0.3" strokeWidth="4" />
                <path d="M22 12a10 10 0 0 0 0-10-10" stroke="currentColor" strokeWidth="4"
strokeLinecap="round" />
              </svg>
            )}
            {loading ? "Analyzing…" : "Analyze Text"}
          </button>
        </div>
      </div>
    </section>
  );
}
```

## `frontend/src/components/ResultCard.jsx`

2.8 KB · 75 lines

Displays the outcome: the predicted class, two probability bars and any error. It explains the threshold and always shows the disclaimer that results are estimates, not proof of authorship.

```
import ProbabilityBar from "./ProbabilityBar";

const DISCLAIMER = "These are machine-learning estimates, not proof of authorship.";

export default function ResultCard({ result, error }) {
  const isAI = result?.label === "AI-Generated";

  return (
    <section className="card h-full" aria-labelledby="result-heading" aria-live="polite">
      <h2 id="result-heading" className="text-lg font-semibold text-white">
        Result
      </h2>

      {error && (
        <div role="alert" className="mt-4 rounded-xl border border-rose-400/30 bg-rose-500/10 p-3 text-sm
text-rose-200">
          {error}
        </div>
      )}

      {!error && !result && (
        <p className="mt-4 text-sm text-slate-400">
          Paste some text and press <span className="text-slate-200">Analyze Text</span> to see the model's
          prediction.
        </p>
      )}

      {result && !error && (
        <div className="mt-4 space-y-5">
          <div>
```

```
            <p className="text-xs uppercase tracking-wider text-slate-400">Predicted class</p>
            <p
              className={`mt-1 inline-flex rounded-xl px-3 py-1.5 text-xl font-bold ring-1 ${
                isAI
                  ? "bg-violet-500/15 text-violet-200 ring-violet-400/30"
                  : "bg-sky-500/15 text-sky-200 ring-sky-400/30"
              }`}
            >
              {isAI ? "AI-Generated" : "Human-Written"}
            </p>
          </div>

          <div className="space-y-4">
            <ProbabilityBar
              label="Estimated Human probability"
              value={result.human_probability}
              gradient="from-sky-400 to-cyan-300"
            />
            <ProbabilityBar
              label="Estimated AI probability"
              value={result.ai_probability}
              gradient="from-indigo-400 to-violet-400"
            />
          </div>

          <p className="text-xs text-slate-400">
            Probabilities come from the model's <code className="text-slate-300">predict_proba()</code>. They show how
            confident the classifier is, not what share of the text was written by AI.
            {result.ai_threshold != null && (
              <>
                {" "}Text is labelled AI-Generated only when the AI probability is at least{" "}
                <span className="font-medium text-slate-200">{Math.round(result.ai_threshold * 100)}%</span> (the
                model's tuned decision threshold).
              </>
            )}
          </p>
        </div>
      )}

      <p className="mt-5 rounded-xl border border-white/10 bg-white/5 p-3 text-xs text-slate-300">
        <span className="font-semibold text-slate-100">Disclaimer: </span>
        {DISCLAIMER}
      </p>
    </section>
  );
}
```

## `frontend/src/components/ProbabilityBar.jsx`

854 bytes · 26 lines

A reusable bar with a label and percentage. `ResultCard` uses it twice. It includes accessibility attributes so screen readers can announce its value.

```
import { formatPercent } from "../lib/text";

export default function ProbabilityBar({ label, value, gradient }) {
  const percent = formatPercent(value);
  return (
    <div>
      <div className="mb-1.5 flex items-baseline justify-between text-sm">
        <span className="text-slate-300">{label}</span>
        <span className="font-semibold tabular-nums text-white">{percent}</span>
      </div>
      <div
        className="h-3 w-full overflow-hidden rounded-full bg-white/10"
        role="progressbar"
        aria-label={label}
        aria-valuemin={0}
```

```
        aria-valuemax={100}
        aria-valuenow={Math.round(value * 100)}
      >
        <div
          className={`h-full rounded-full bg-gradient-to-r ${gradient} transition-[width] duration-700 ease-out`}
          style={{ width: percent }}
        />
      </div>
    </div>
  );
}
```

## frontend/src/components/HistoryPanel.jsx

2.0 KB · 48 lines

Lists this session's analyses with their label, time, a text preview and probabilities. Clicking an entry shows its result again, and a button clears the list.

```
import { formatPercent } from "../lib/text";

export default function HistoryPanel({ history, onSelect, onClear }) {
  return (
    <section className="card" aria-labelledby="history-heading">
      <div className="flex items-center justify-between gap-3">
        <div>
          <h2 id="history-heading" className="text-lg font-semibold text-white">
            Session history
          </h2>
          <p className="text-xs text-slate-400">Stored only in this browser tab; cleared when the session ends.</p>
        </div>
        {history.length > 0 && (
          <button type="button" className="btn-ghost px-3 py-1.5 text-xs" onClick={onClear}>
            Clear history
          </button>
        )}
      </div>

      {history.length === 0 ? (
        <p className="mt-4 text-sm text-slate-400">No analyses yet.</p>
      ) : (
        <ul className="mt-4 grid max-h-96 gap-2 overflow-y-auto pr-1 md:grid-cols-2">
          {history.map((item) => (
            <li key={item.id}>
              <button
                type="button"
                onClick={() => onSelect(item)}
                className="h-full w-full rounded-xl border border-white/5 bg-white/[0.03] p-3 text-left transition hover:border-indigo-400/30 hover:bg-white/[0.06]"
              >
                <div className="flex items-center justify-between gap-2 text-xs">
                  <span className={`font-semibold ${item.label === "AI-Generated" ? "text-violet-300" : "text-sky-300"}`}>
                    {item.label}
                  </span>
                  <span className="text-slate-500">{new Date(item.time).toLocaleTimeString()}</span>
                </div>
                <p className="mt-1 line-clamp-2 break-words text-sm text-slate-300">{item.text}</p>
                <p className="mt-1 text-xs tabular-nums text-slate-500">
                  Human {formatPercent(item.human_probability)} · AI {formatPercent(item.ai_probability)}
                </p>
              </button>
            </li>
          ))}
        </ul>
      )}
    </section>
```

```
  );
}
```

## frontend/src/components/AboutModel.jsx

3.1 KB · 67 lines

Explains the model in plain English in four steps: cleaning, TF-IDF, Logistic Regression and reading the probabilities. When the model is loaded, `describeSteps()` also shows the real pipeline structure and threshold from the backend.

```
const STEPS = [
  {
    title: "1. Cleaning the text",
    body: "Before anything else, the text is lower-cased, curly quotes are turned into straight ones, and extra spaces and line breaks are collapsed. This is the same clean_text() function used during training, so the model sees text in the format it learned from.",
  },
  {
    title: "2. TF-IDF turns text into numbers",
    body: "TF-IDF (Term Frequency – Inverse Document Frequency) scores each feature: high when it appears often in this text but is rare across the dataset, low when it is everywhere. The model combines two views: words and two-word phrases (up to 100,000 features, punctuation included), and 3–5 letter character chunks (up to 150,000 features), which capture spelling, word endings and style.",
  },
  {
    title: "3. Logistic Regression makes the decision",
    body: "Logistic Regression learns a weight for every feature. Features seen more often in AI-generated essays push the score up; features more common in human essays push it down. The sum is squashed into a 0–1 probability with the sigmoid function. Text is only labelled AI when that probability reaches the tuned threshold, which reduces false accusations against human writers.",
  },
  {
    title: "4. Reading the probabilities",
    body: "The two bars come from predict_proba(). They describe how confident the model is, based on patterns in its training data. They are not the percentage of the text written by AI, and they can be wrong, especially for short texts, unusual writing styles, or topics unlike the training essays.",
  },
];

function describeSteps(steps) {
  return Object.entries(steps)
    .map(([name, type]) =>
      typeof type === "string"
        ? `${name} (${type})`
        : `${name} (${Object.entries(type)
            .map(([sub, t]) => `${sub}: ${t}`)
            .join(" + ")})`
    )
    .join(" → ");
}

export default function AboutModel({ health }) {
  const data = health.data?.model_loaded ? health.data : null;

  return (
    <section className="card" aria-labelledby="about-heading">
      <h2 id="about-heading" className="text-lg font-semibold text-white">
        About the model
      </h2>
      <p className="mt-1 text-sm text-slate-400">
        An improved scikit-learn pipeline trained on the AI vs Human essay dataset and loaded by the backend with
        joblib.
      </p>

      <div className="mt-5 grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {STEPS.map((step) => (
          <div
            key={step.title}
            className="rounded-xl border border-white/10 bg-gradient-to-b from-white/[0.06] to-transparent p-4"
```

```
          >
            <h3 className="text-sm font-semibold text-indigo-200">{step.title}</h3>
            <p className="mt-2 text-sm leading-relaxed text-slate-300">{step.body}</p>
          </div>
        ))}
      </div>

      {data && (
        <div className="mt-4 space-y-1 break-words text-xs text-slate-500">
          <p>Loaded pipeline: {describeSteps(data.pipeline_steps)}</p>
          <p>
            AI decision threshold: {Math.round(data.ai_threshold * 100)}% · Minimum length: {data.min_words}
words
          </p>
        </div>
      )}
    </section>
  );
}
```

## `frontend/src/lib/api.js`

1.1 KB · 41 lines

All communication with the backend is in this one file.

- `request()` sends a request and turns network failures into a helpful message.
- `readError()` extracts the readable `detail` message from an error response.
- `getHealth()` calls `GET /health` and `predictText()` calls `POST /predict`.

```
const API_URL = (import.meta.env.VITE_API_URL || "http://localhost:8000").replace(/\/+$/, "");

export { API_URL };

/** Turn a FastAPI error body into a readable message. */
async function readError(response) {
  try {
    const body = await response.json();
    if (typeof body.detail === "string") return body.detail;
    if (Array.isArray(body.detail)) return body.detail.map((d) => d.msg).join(" ");
  } catch {
    /* body was not JSON */
  }
  return `Request failed with status ${response.status}.`;
}

async function request(path, options = {}) {
  let response;
  try {
    response = await fetch(`${API_URL}${path}`, options);
  } catch {
    throw new Error(
      `Cannot reach the backend at ${API_URL}. Make sure the FastAPI server is running (see README).`
    );
  }
  if (!response.ok) throw new Error(await readError(response));
  return response.json();
}

export function getHealth() {
  return request("/health");
}

export function predictText(text, signal) {
  return request("/predict", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ text }),
    signal,
```

```
  });
}
```

## frontend/src/lib/samples.js

1.8 KB · 16 lines

Holds the two sample texts behind the sample buttons. Both were taken from the dataset, along with its human or AI label.

```
// Short examples taken from AI_Human_cleaned.csv, with the dataset's own `generated` label.
// They come from the training data, so the model is expected to be confident on them.
export const SAMPLES = [
  {
    id: "human",
    name: "Dataset sample: labelled human",
    text: `People should not be able to use cell phone while they are driving in any capacity. The reasons they should not be allowed to use them is because they could seriously hurt someone or kill them when they are using their cell phones. When you are using a cell phone while operating a vehicle you are paying attention to the phone rather than the road.

Let's say you were using your phone and you hit somebody and killed them what would you do. You would probably drive off and then call the police but if you did do that then they would check traffic cameras and the get your license plate and you would go to jail for a hit and run charge. But if you stay then you're going to either get a ticket for distracted driving or you could go to jail on a manslaughter charge so its up to you on what you do.`,
  },
  {
    id: "ai",
    name: "Dataset sample: labelled AI",
    text: `I mean, yeah, talking to multiple people for advice can definitely help. I guess it's because when you hear different perspectives and ideas, you get a more balanced view on the situation. Plus, everyone has different experiences and knowledge, so they might bring up something you hadn't even thought about. Like, let's say you're trying to decide which college to go to. If you only ask one person, they might just tell you to go to their alma mater because they had a good time there. But if you ask a few people, you'll hear about different schools and what they liked or didn't like about them. Then you can weigh all those factors and make a more informed decision.`,
  },
];
```

## frontend/src/lib/text.js

215 bytes · 8 lines

Two helpers: `countWords()` counts words and `formatPercent()` turns 0.783 into 78.3%.

```
export function countWords(text) {
  const trimmed = text.trim();
  return trimmed ? trimmed.split(/\s+/).length : 0;
}

export function formatPercent(probability) {
  return `${(probability * 100).toFixed(1)}%`;
}
```

## frontend/src/lib/useSessionHistory.js

810 bytes · 31 lines

A custom React hook that stores up to 20 past analyses in `sessionStorage`, so history survives a page refresh but disappears when the tab closes. If storage is blocked, history still works in memory.

```
import { useEffect, useState } from "react";

const KEY = "ai-human-classifier:history";
const MAX_ITEMS = 20;

function load() {
  try {
    const raw = sessionStorage.getItem(KEY);
```

```
    return raw ? JSON.parse(raw) : [];
  } catch {
    return [];
  }
}

/** Analysis history kept in sessionStorage, so it lasts only for this browser tab/session. */
export function useSessionHistory() {
  const [history, setHistory] = useState(load);

  useEffect(() => {
    try {
      sessionStorage.setItem(KEY, JSON.stringify(history));
    } catch {
      /* storage unavailable (private mode etc.) - history still works in memory */
    }
  }, [history]);

  const add = (entry) => setHistory((prev) => [entry, ...prev].slice(0, MAX_ITEMS));
  const clear = () => setHistory([]);

  return { history, add, clear };
}
```

# 8. docs: Project Documents

These are documents, so no code is printed. This guide is saved here too, as `docs/Project_File_Guide.pdf`.

### `docs/MLE_PBL_Project_Instructions.pdf`

143.9 KB

The instructor's project brief, which sets the required pipeline, report structure and deadlines.

### `docs/AI_vs_Human_Text_Classifier.pptx`

458.4 KB

The current presentation, which covers the final model only.

### `docs/pitch.md`

5.3 KB

The content of each slide in Markdown. It can be pasted into a slide generator.

### `docs/AI_vs_Human_Text_Classifier_Report.pdf`

1003.4 KB

The detailed 24-page project report with benchmarks, following the structure in the brief.

### `docs/AI_vs_Human_Text_Classifier_Report.html`

474.2 KB

The same report as a single HTML file that opens in a browser and includes a Print button.

### `docs/archive/AI_vs_Human_Text_Classifier_full_23_slides.pptx`

542.5 KB

The earlier 23-slide deck, which includes the older models and the comparison the brief asks for.

### `docs/archive/pitch_full_with_viva_QA.md`

20.4 KB

The earlier long pitch. It is worth keeping for its viva questions and answers.

# 9. What Happens When You Click Analyze

This follows one prediction through the files. It is also a useful way to explain the project in a viva.

1. **Starting up.** `run.bat` starts both servers. In `main.py`, `lifespan()` calls `ModelService.load()`, which reads the paths from `config.py` and `.env`, registers `clean_text` and loads the joblib model.
2. **Opening the page.** `App.jsx` calls `getHealth()` in `api.js`. `GET /health` replies with the limits and model status, and `Header.jsx` shows Model loaded.
3. **Typing.** `TextInput.jsx` updates the text in `App.jsx` and shows the counts from `countWords()` in `text.js`. Analyze Text is enabled at 50 words.
4. **Clicking Analyze Text.** `handleAnalyze()` in `App.jsx` calls `predictText()` in `api.js`, which sends `POST /predict` with the text as JSON.
5. **Checking the input.** In `main.py`, `PredictRequest.validate_text()` rejects empty or over-long text, then `predict()` checks the minimum word count. Problems return HTTP 422 with a readable message.
6. **Running the model.** `ModelService.predict()` calls `predict_proba()` on the pipeline. Inside it, `clean_text()` cleans the text, the word and character TF-IDF steps create 250,000 features, and Logistic Regression produces the two probabilities.
7. **Choosing the label.** If the AI probability is at least 0.65, the label is AI-Generated. Otherwise it is Human-Written.
8. **Replying.** `main.py` returns the label, both probabilities, the threshold and the disclaimer as JSON.
9. **Showing the result.** `ResultCard.jsx` shows the class and two `ProbabilityBar.jsx` bars, and `useSessionHistory.js` adds the result to `HistoryPanel.jsx`.